

Ejercicio Nro: 13

Enunciado:

- 1- ¿Qué es Extreme Programming (XP) y cuál es su objetivo principal dentro de las metodologías ágiles?
- 2- ¿Cuáles son los cinco valores principales de XP? Explicá brevemente cada uno.
- 3- ¿Por qué XP considera que las pruebas son un elemento fundamental del desarrollo de software?
- 4- ¿Qué es Test Driven Development (TDD) y cómo se relaciona con XP?
- 5- ¿En qué consiste la práctica de Pair Programming? Menciona dos ventajas y una posible dificultad.
- 6- ¿Qué son las historias de usuario en XP y por qué se prefieren frente a una especificación extensa de requisitos?
- 7- ¿Qué significa Continuous Integration en XP y qué beneficios aporta al equipo de desarrollo?
- 8- ¿Cómo se aplica el concepto de Weekly Cycle en un proyecto desarrollado con XP?
- 9- En XP se plantea que se fija el tiempo, el costo y la calidad, y se negocia el alcance. ¿Qué significa esta idea? Explicalo con un ejemplo.
- 10- Elegí tres prácticas de XP y explicá cómo podrían aplicarse en un proyecto real de desarrollo de software.

Resolución:

1.

Es un enfoque de desarrollo de software que se centra en un proceso adaptativo y orientado a las personas.

Objetivo principal: Sus promesas y objetivos principales son reducir el riesgo del proyecto, mejorar la respuesta ante los cambios en el negocio, mejorar la productividad a lo largo de la vida del software y hacer más divertido el trabajo del equipo construyendo el software.

2.

Los 5 valores fundamentales que guían la metodología son:

- **Comunicación:** Fomenta el intercambio constante de información dentro del equipo y con el cliente.
- **Simplicidad:** Se busca diseñar y programar solo lo que es necesario en el momento, evitando complejidades innecesarias.
- **Retroalimentación (Feedback):** Aprender constantemente de las pruebas, del sistema y del cliente para realizar ajustes inmediatos.

- **Valentía (Coraje):** Capacidad para tomar decisiones firmes, como aceptar cambios, rehacer código si es necesario o enfrentar problemas complejos sin miedo.
- **Respeto:** Valorar a los miembros del equipo, sus aportes y asegurar que todos se sientan respetados en su trabajo.

3.

XP sostiene que las pruebas son un pilar básico porque sirven para dar certidumbre y asegurar la calidad del código. El documento destaca que:

- Son el elemento fundamental del desarrollo y **todos los desarrolladores deben escribirlas** mientras escriben el código que finalmente irá a producción.
- Se integran en un proceso de integración y construcción continua, lo que genera una plataforma muy estable para el desarrollo futuro.
- Ofrecen confianza y ritmo de trabajo; si un test es difícil de escribir, suele ser una señal de que hay un problema de diseño (código acoplado).

4.

- **Qué es:** Es el desarrollo guiado por pruebas (también llamado *Test-First Programming*), una práctica donde se escriben las pruebas automáticas **antes** de escribir cualquier línea de código de producción.
- **Relación con XP:** TDD es una de las prácticas esenciales (primarias) de XP. Sigue un ciclo muy marcado de tres pasos: escribir una prueba que falle (**Red**), escribir el código mínimo para que la prueba pase (**Green**), y finalmente limpiar y optimizar el código (**Refactor**). Kent Beck, creador de XP, es además una de las figuras clave en el desarrollo de esta práctica y de herramientas como JUnit.

5.

- **En qué consiste:** Todo el código que va a producción debe ser escrito por dos personas sentadas frente a la misma máquina. Es un diálogo constante entre dos programadores que están simultáneamente analizando, diseñando, probando e intentando programar mejor.
- Ventajas:
 - 1-Se mantienen centrados mutuamente.
 - 2-Se cumplen mejor los estándares del equipo.
- **Posible dificultad:** El documento menciona como consejo/advertencia "No invadir el espacio personal del otro (monitores grandes)", lo que refleja que la cercanía física y la falta de espacio adecuado pueden generar incomodidad o fricciones. También advierte que no se deben juntar dos programadores novatos.

6.

- **Qué son:** Son descripciones breves de funcionalidades escritas en tarjetas pequeñas, que incluyen un nombre, una descripción y una estimación de tiempo.
- **Por qué se prefieren:** Se prefieren porque el término "requisitos" tiene una connotación de "inmutabilidad" y "permanencia" que choca con la filosofía ágil de "abrazar el cambio". Al usar historias en tarjetas pequeñas (y colocarlas a la vista en la pared, no en un programa), es mucho más fácil estimar el coste de cada una, moverlas, priorizarlas y negociar el alcance de forma flexible.

7.

Significa no dejar pasar más de dos horas sin integrar en la base de código común los cambios que los programadores han desarrollado. Cada pareja, tras un par de horas de trabajo, sube sus cambios para que se complete el "build" (construcción) y se ejecuten todos los tests automáticos.

- **Beneficios:**
 - Rompe el problema de "divide, vencerás e integrarás" (evita que la integración se vuelva un caos impredecible al final).
 - Asegura que no haya problemas de regresión (que algo nuevo rompa lo que ya funcionaba).
 - Permite detectar errores de forma inmediata; si el build diario falla o se "notifica con alertas", el equipo se entera al instante para solucionarlo.
 - Mantiene el sistema siempre listo para ser desplegado en producción.

8.

El ciclo semanal se aplica como el motor del progreso a corto plazo mediante los siguientes pasos:

1. **Reunión al comienzo de cada semana:** Se revisa el progreso hasta la fecha (incluyendo si el progreso real se corresponde con lo planificado).
2. **Selección de Historias:** El cliente elige qué historias de usuario (que sumen el equivalente a una semana de trabajo) se van a realizar en esos días.
3. **Fraccionamiento y Estimación:** Los desarrolladores dividen esas historias en tareas técnicas más chicas, se las reparten y las estiman.
4. **Desarrollo y Pruebas:** Se comienza la semana escribiendo las pruebas automáticas de las historias y se progresa implementándolas.
5. **Fin del ciclo:** Al terminar la semana, las nuevas historias tienen que estar completamente listas para ser desplegadas.

9.

En el "triángulo de hierro" tradicional (tiempo, coste, alcance), bajar la calidad para cumplir los plazos es una pésima decisión porque genera "deuda técnica". XP propone que el Tiempo de entrega, el Costo (recursos/equipo) y la Calidad del software no se tocan (son fijos). Por lo tanto, si el tiempo aprieta, lo único que se puede ajustar es el **Alcance** (es decir, qué tantas cosas o funcionalidades se van a entregar en esa fecha).

- **Ejemplo:** Imaginemos que estamos desarrollando una tienda online y la fecha de lanzamiento (Tiempo) es en dos semanas, el presupuesto y los programadores (Costo) están definidos, y el sistema debe funcionar perfectamente sin errores (Calidad). Si vemos que no llegamos a programar todo, no compaginamos el código a las apuradas (lo que rompería la calidad); en su lugar, nos reunimos con el cliente y negociamos: *"En dos semanas lanzamos la web con el carrito y pagos con tarjeta (Alcance cubierto), pero dejamos la sección de cupones de descuento y pagos con criptomonedas para la siguiente entrega (Alcance postergado)"*.

10.

1. Informative Workspace (Espacio de trabajo informativo):
 - *Aplicación real:* En la oficina del equipo de desarrollo, se destina una pizarra física o pared principal (visible para todos) donde se pegan tarjetas (post-its) divididas en columnas como *"Por hacer"*, *"Esta semana"*, *"Hecho"*. Además, se cuelga un monitor que muestre un gráfico de barras con la evolución del proyecto en tiempo real. Así, cualquiera que pase sabe exactamente el estado actual del software.
2. Ten-Minute Build (Construcción en 10 minutos):
 - *Aplicación real:* En un proyecto web, se configura un script automatizado (como en GitHub Actions). Cada vez que un desarrollador termina una tarea, sube su código y el servidor automáticamente compila el proyecto, levanta la base de datos de prueba y ejecuta los cientos de tests en menos de 10 minutos. Si algo se rompe, el sistema avisa enseguida y el equipo lo corrige antes de avanzar.
3. Energized Work (Trabajo energizado):
 - *Aplicación real:* Para evitar que los desarrolladores se quemen y escriban código defectuoso por cansancio, el líder de proyecto prohíbe las horas extras sistemáticas y fomenta la jornada estándar de 40 horas. En el día a día, se implementa una regla: de 14:00 a 16:00 hs se declara "Tiempo de Programación enfocado", donde todos apagan las notificaciones de Slack, no ven emails y se concentran al 100% solo en tirar código sin interrupciones.